

Ejercicio 6: Colisiones

Partiendo del código generado en la práctica 5, se pide crear un programa que muestre una ventana de OpenGL con al menos un objeto en pantalla, con el que se puedan detectar colisiones.

Clase Collider y ColliderTree:

Incorporar las clases Collider y ColliderTree vistas en clase para manejar colisiones entre objetos.

Clase Sphere:

La clase Collider representa el tipo “colisionador” base, que se usará para implementar la detección de colisiones. Sobre esa base, se pide implementar la clase “Sphere”, heredera de Collider, que detectará colisiones usando las propiedades de esferas.

- Las esferas están representadas por un radio y un centro.

Clase Object3D:

Añadir un atributo “Collider* coll;” que representará el colisionador de este objeto. Añadir un atributo “tipo” (se da libertad para implementarlo) que permita distinguir objetos de “distinto tipo”, como se vio en clase.

Añadir un atributo “bool active” que representa que un objeto está activo en el sistema. Cuando esta variable valga “false”, el objeto pasará a estar activo y acabará borrado del sistema.

Modificar el método de carga de mallas para que, una vez cargada la información del objeto, se cree su objeto collider de tipo esfera.:

- Dada una lista de triángulos, se creará una partícula por cada triángulo. El centro estará lo más cerca posible del centro geométrico del volumen descrito por esas partículas (media de los máximos y mínimos en XYZ de todos los vértices).
- Se creará un colisionador de esferas: El radio se calculará de manera que el trazado del perímetro de la esfera desde ese centro deberá contener todos los vértices.

Clase Render:

Realizar los cambios necesarios para poder obtener listas de colisiones, destruir objetos o añadir objetos al sistema. Métodos nuevos:

- Método para obtener todos los objetos de un tipo "objType" que colisionen con el objeto "obj" pasados por parámetros:

```
vector<Object3D*> getCollisions(Object3D* obj, int objType)
```
- Métodos para quitar objetos del sistema. Se deben destruir para liberar memoria una vez se dejen de usar:
- ```
void deleteObject(Object3D* obj)
```

Modificar el método "mainLoop" de la clase Render para que elimine de forma adecuada los objetos que no se encuentren activos.

## Clase Camera:

Añadir un atributo "Collider\* coll", que simulará el colisionador de la cámara. Será creado usando un único triángulo/partícula de tamaño 0.25\*0.25\*0.25 unidades, que representará el tamaño de la cámara. Modificar el método "step" de la cámara para que, una vez movida, se actualice el colisionador a la nueva posición de la cámara.

Modificar también el método step para que detecte colisión con objetos de la escena. En caso de que haya colisión, la cámara deberá retroceder "n" unidades hasta su posición original "antes" de la colisión (guardar la posición previa al inicio del método step, mover la cámara y detectar colisión. Si la hay, se vuelve a la posición guardada).

## Programa principal:

- Usando las clases creadas anteriormente, se pide implementar un programa que interactúe con el usuario por teclado y ratón, y que realice las siguientes operaciones:
- **Parte 1:**
- Cargar un objeto de tipo cubo3D como el visto en clase, y posicionarlo en las coordenadas 0,0,0.
- Añadir un punto de luz en la posición 3,3,3
- Añadir la cámara en la posición 0,0,3
- Al mover la cámara, detectar colisión con el cubo. Si hay colisión, la cámara no lo atravesará.

## Evaluación:

La práctica se considerará aprobada si al menos carga un archivo en formato vertc como el usado en clase (un cuadrado con dos triángulos), con una textura simple y que al mover la cámara no lo atraviese. Además, deberá usar los shaders vistos en clase para renderizar el escenario. A continuación, se detallan las partes que se consideran “extras” para llegar al 10:

- Aprobado (5 puntos):
  - Implementación del sistema de colisiones visto en clase
    - El cubo está dentro de una esfera
    - La cámara está dentro de otra esfera
    - Sólo se mueve la cámara (el cubo está quieto), su esfera colisionadora la sigue
    - Al detectar colisión con la esfera del cubo, la cámara se para
- +5 puntos (escenario adicional al anterior, +5 si se implementa “todo”):
  - +3 puntos:
    - Implementar un juego 2D, basado en sprites. En función de la lógica implementada, clases implementadas, sprites, “complejidad”, etc... Sumará hasta 3 puntos
  - +2 puntos:
    - Uso de efectos de luz (varias luces), mezcla de color, etc..

## Entrega:

Se debe entregar el código desarrollado a través del blackboard antes de la fecha de entrega dada. Si se entrega más tarde, la práctica tendrá una penalización del 50% del total de la puntuación. En caso de trabajar en grupo ambos alumnos deben entregar la práctica a través del blackboard.

Nombrado del fichero:

Entregar un fichero en formato zip siguiendo la siguiente nomenclatura:

- PRG6\_NOMBREAPELLIDO1\_NOMBREAPELLIDO2.zip

La práctica debe defenderse ante el profesor. Si se realiza en parejas, ambos alumnos deben estar presentes. Las fechas de defensa se comunicarán una vez realizadas las entregas.